

I. Лабораторная работа «Загружаемые модули ядра»

Доц. Рязанова Н.Ю.

Цель работы: знакомство со структурой загружаемого модуля ядра принципами их разработки.

Теоретическая часть

Макросы `module_init` и `module_exit`

Загружаемые модули ядра должны содержать два макроса **`module_init`** и **`module_exit`**.

Макрос `module_init`

Служит для регистрации функции инициализации модуля. Макрос принимает имя функции в качестве фактического параметра. В результате эта функция будет вызываться, при загрузке модуля в ядро. Передаваемая функция должна соответствовать следующему прототипу:

```
int func_init(void);
```

Функция возвращает значение типа `int`. Если функция инициализации завершилась успешно, то возвращается значение ноль. В случае ошибки возвращается ненулевое значение.

Как правило, функция инициализации предназначена для запроса ресурсов и выделения памяти под структуры данных и т.п.. Так как функция инициализации редко вызывается за пределами модуля, ее обычно не нужно экспортировать и можно объявить с ключевым словом `static`.

Если функция не передается в макрос, то код, который необходимо выполнить при загрузке модуля может быть написан в самом макросе.

Определение макроса `module_init`:

```
#define __initcall(fn) \
static initcall_t __initcall_##fn __init_call = fn
#define __init_call __attribute__ \
((unused, __section__ ("function_ptrs")))
#define module_init(x) __initcall(x);
```

Макрос `module_exit`

Макрос служит для регистрации функции, которая вызывается при удалении модуля из ядра. Обычно эта функция выполняет задачу освобождения ресурсов. После завершения функции модуль выгружается.

Функция завершения должна соответствовать прототипу:

```
void func_exit(void);
```

Функцию завершения, как и инициализации, можно объявить как static.

Пример простого загружаемого модуля ядра:

```
static int __init md_init( void )
{
    printk( "Module md loaded!\n" );
    return 0;
}
```

```
static void __exit md_exit( void )
{
    printk( "Module unloaded!\n" );
}
```

```
module_init( md_init );
module_exit( md_exit );
```

Комментарии к примеру:

В предложенном коде используются макросы __init, __exit а также функция printk. Макрос __init используются для обозначения функций, встраиваемых в ядро, или инициализированных данных, которые ядро воспринимает как указание о том, что функция используется только на этапе инициализации и освобождает использованные ресурсы после. Это позволяет экономить ресурсы системы.

В ядре макрос определяется следующим образом:

```
#define __init __section(.init.text) \
__cold __latent_entropy __noinitretpoline
```

The __exit macro causes the omission of the function when the module is built into the kernel, and like __exit, has no effect for loadable modules. Again, if you consider when the cleanup function runs, this makes complete sense; built-in drivers don't need a cleanup function, while loadable modules do.

В ядре макрос определяется следующим образом:

```
#define __exit __section(.exit.text) \
__exitused __cold notrace
```

Макросы `__init` и `__exit` определены в `linux/init.h`.
<https://elixir.bootlin.com/linux/v3.14/source/include/linux/init.h>

Если файл будет статически скомпилирован с образом ядра, то данная функция не будет включена в образ и никогда не будет вызвана (так как если нет модуля, то код никогда не может быть удален из памяти).

Функция `printk` определена в ядре Linux и доступна модулям. Функция ведёт себя аналогично библиотечной функции `printf`. Став частью ядра модуль не может вызывать обычные библиотечные функции, поэтому ядро предоставляет модулю функцию `printk`.

Функция `printk` позволяет отправлять сообщения в системный журнал. Сама функция не производит запись в системный журнал, а записывает сообщение в специальный буфер ядра. Из буфера ядра записанные сообщения могут быть прочитаны демоном протоколирования.

Пример использования функции `printk` :

```
int var = 1;
printk(KERN_INFO "var = %d\n", var);
```

где `KERN_INFO` – уровень протоколирования сообщения.

Система поддерживает восемь уровней протоколирования, показанных в таблице 1. Все константы определены в файле `linux/kernel.h`.

Таблица 1

Уровень	Константа	Описание
7	KERN_DEBUG	Отладочные сообщения, самый низкий приоритет
6	KERN_INFO	Информационные сообщения
5	KERN_NOTICE	Это уже не информационное сообщение, но еще и не предупреждение
4	KERN_WARNING	Предупреждение: скоро может пойти что-то не так
3	KERN_ERR	Возникла ошибка
2	KERN_CRIT	Возникла критическая ошибка
1	KERN_ALERT	Тревога — система скоро "развалится"
0	KERN_EMERG	Система "развалилась" (система больше не может использоваться)

Стоит обратить внимание на то, что после уровня протоколирования не стоит запятая, так как она является частью строки форматирования, это сделано в системе для экономии памяти стека при вызове функции.

Макросы для установки информации о модуле

Макросы `MODULE_LICENSE`, `MODULE_AUTHOR`, `MODULE_DESCRIPTION`, `MODULE_SUPPORTED_DEVICE`, `MODULE_VERSION` — это макросы, которые «дают» linux информацию о модуле, которую потом можно получить с помощью команды `modinfo`. Описание макросов содержится в файле `linux/module.h`.

Макрос `MODULE_LICENSE`

Макрос используется для того, чтобы сообщить ядру, под какой лицензией распространяется исходный код модуля, что влияет на то, к функциям и переменным (символы ядра) он может получить доступ в ядре. Этот макрос является обязательным. Например, модуль под лицензией GPLv2 имеет доступ ко всем символам ядра. Без этой декларации при загрузке модуля ядро выводит предупреждение.

Поддерживаемые варианты:

Параметр	Значение
"GPL"	[GNU Public License v2 or later]
"GPL v2"	[GNU Public License v2]
"GPL and additional rights"	[GNU Public License v2 rights and more]
"Dual BSD/GPL"	[GNU Public License v2 or BSD license choice]
"Dual MIT/GPL"	[GNU Public License v2 or MIT license choice]
"Dual MPL/GPL"	[GNU Public License v2 or Mozilla license choice]
"Proprietary"	[Non free products]

Загрузка в память модуля, для которого лицензия не соответствует GPL, приведет к установке в ядре флага `tainted` (испорчено), который служит для информационных целей (например, в сообщениях об ошибках).

Пример использования:

```
MODULE_LICENSE("GPL");
```

Макрос MODULE_AUTHOR

Позволяет указать автора модуля. Значение этого макроса служит только для информационных целей.

Пример использования:

```
MODULE_AUTHOR("Author");
```

Макрос MODULE_DESCRIPTION

Позволяет указать описание модуля. Значение этого макроса служит только для информационных целей.

Макрос MODULE_SUPPORTED_DEVICE

Помещает запись, описывающую, какое устройство поддерживается этим модулем. Комментарии в источниках ядра предполагают, что в конечном итоге эти параметры могут использоваться для автоматической загрузки модулей, но в настоящее время такое использование не производится.

Макрос MODULE_VERSION

Позволяет указать версию модуля. Значение этого макроса служит только для информационных целей.

Экспортируемые символы

Экспортируемыми символами называются данные и функции, которыми могут пользоваться другие модули ядра. При загрузке модули динамически компоуются с ядром, в коде модулей могут вызываться только те функции ядра, которые явно экспортируются для использования.

```
EXPORT_SYMBOL(md1_data);  
EXPORT_SYMBOL(md1_func);
```

Для того, чтобы другие модули могли использовать экспортированные символы, они должны “знать” их определение. Для этого применяются заголовочные файлы.

Макрос EXPORT_SYMBOL_GPL

Иногда необходимо, чтобы символы были доступны только для модулей, имеющих соответствующую лицензию GPL. Для этого используют макрос EXPORT_SYMBOL_GPL().

Команды linux для работы с загружаемыми модулями

В ОС Linux существуют специальные команды для работы с загружаемыми модулями ядра.

insmod

Загружает модуль в ядро из конкретного файла, если модуль зависит от других модулей, которые не загружены в ядро, то выдает ошибку и не загружает модуль. Только суперпользователь может загрузить модуль в ядро.

Пример использования: `sudo insmod ./md.ko`

lsmod

Выводит список модулей, загруженных в ядро.

modinfo

Извлекает информацию из модулей ядра (лицензия, автор, описание и т.д.).

Пример использования:

```
modinfo md.ko
filename:      /home/alexander/Develop/OS/Kernel/TASK1/md.ko
author:        Sychev Svyatoslav
license:       GPL
srcversion:    10E5B7B44C5B0320F3FC7B2
depends:
name:          md
vermagic:      4.13.0-32-generic SMP mod_unload
```

rmmod

Команда используется для выгрузки модуля из ядра, в качестве параметра передается имя файла модуля. Только суперпользователь может выгрузить модуль из ядра.

Пример использования: `sudo rmmod md1.ko`

dmesg

Команда для вывода буфера сообщений ядра в стандартный поток вывода. Сообщения содержат информацию о драйверах устройств, загружаемых в ядро во время загрузки системы, а также при подключении аппаратного обеспечения к системе.

Замечание: системный журнал используется большим числом процессов, и для того чтобы было легче обнаружить сообщения конкретного модуля, рекомендуется в их начало помещать некоторый идентификатор (к примеру знак + или имя модуля). В этом случае просмотреть сообщения от модуля можно при помощи команды:

```
dmesg | tail -n60 | grep +
```

На рис.1 показан кольцевой список процессов struct list_head:

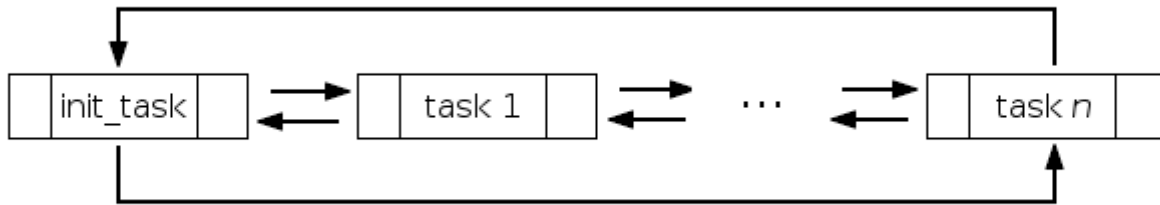


Рис.1

Следующие примеры демонстрируют работу с о связным списком процессов:

- В linux/sched.h определен макрос for_each_process():

```
#define for_each_process(p) \
    for (p = &init_task ; (p = next_task(p)) != &init_task ; )
```

- next_task() – макрос, определенный в linux/sched.h, который возвращает дескриптор следующего процесса с списке:

```
#define next_task(p) list_entry((p)->tasks.next, struct task_struct, tasks)
```

Пример загружаемого модуля ядра из книги Robert Love Chapter 3:

```
/* ProcessList.c
   Robert Love Chapter 3
*/
#include < linux/kernel.h >
#include < linux/sched.h >
#include < linux/module.h >

int init_module(void)
{
    struct task_struct *task;
    for_each_process(task)
    {
        printk("%s pid - %d\n", task->comm , task->pid);
    }

    return 0;
}

void cleanup_module(void)
{
    printk(KERN_INFO "Cleaning Up.\n");
}
```

task_struct

В ядре Linux каждый процесс и поток описывается с помощью структуры task_struct. Действительно, в Linux поток считается стандартным процессом, который совместно использует определенные ресурсы (например, таблицы страниц и обработчики сигналов) с другими процессами.

Структура task_struct определена в файле linux/sched.h. В текущей версии ядра (6.9.6) она начинается со строки 748:

<https://elixir.bootlin.com/linux/v6.9.6/source/include/linux/sched.h#L748>

В этой структуре много полей, некоторые из которых зависят от параметров конфигурации ядра. Например, параметр конфигурации ядра CONFIG_SMP включает поддержку симметричной многопроцессорной обработки (то есть поддержку компьютеров с несколькими центральными процессорами), что добавляет в структуру task_struct необходимые поля:

task_struct

В ядре Linux каждый процесс и поток описывается с помощью структуры task_struct. Действительно, в Linux поток считается стандартным процессом, который совместно использует определенные ресурсы (например, таблицы страниц и обработчики сигналов) с другими процессами.

Структура task_struct определена в файле linux/sched.h. В текущей версии ядра (6.9.6) она начинается со строки 748:

<https://elixir.bootlin.com/linux/v6.9.6/source/include/linux/sched.h#L748>

В этой структуре много полей, некоторые из которых зависят от параметров конфигурации ядра. Например, параметр конфигурации ядра CONFIG_SMP включает поддержку симметричной многопроцессорной обработки (то есть поддержку компьютеров с несколькими центральными процессорами), что добавляет в структуру task_struct необходимые поля:

Необходимо обратиться к документации, чтобы узнать обо всех доступных полях. Вот основные из них, которые я буду использовать в своем пользовательском модуле ядра:



pid на самом деле вводит в заблуждение: поскольку ядро Linux хранит структуры task_struct для каждого потока, pid на самом деле является идентификатором потока;

tgid (идентификатор группы задач) — это фактический PID: вы также можете определить это поле как pid основного потока каждого процесса;

comm — это имя исполняемого файла без пути, длина которого (обычно) не превышает 16 символов.

Дерево процессов

Структура task_struct содержит различные поля, которые позволяют отслеживать дерево процессов и перемещаться по нему:

task_struct *parent — указатель на структуру task_struct родительского процесса; list_head children — двусвязный список дочерних процессов; list_head sibling: процессы, которые являются прямыми потомками одного и того же родительского процесса, хранятся в двусвязном списке sibling.

The current macro - текущий макрос — это ссылка на дескриптор процесса (указатель на task_struct) выполняемого в данный момент процесса. Способ выполнения текущей задачи зависит от архитектуры. В архитектуре x86 это делается с помощью функции current_thread_info() в asm/thread_info.h.

Наконец, current разыменовывает элемент task структуры thread_info, которая воспроизведена ниже из asm/thread_info.h: current_thread_info()->task;

```
struct thread_info {
    struct task_struct    *task;        /* main task structure */
    struct exec_domain    *exec_domain; /* execution domain */
    unsigned long         flags;        /* low level flags */
    unsigned long         status;       /* thread-synchronous flags */
    __u32                 cpu;          /* current CPU */
    int                   preempt_count; /* 0 => preemptable, <0 => BUG */

    mm_segment_t          addr_limit;   /* thread address space:
                                         0-0xBFFFFFFF for user-thread
                                         0-0xFFFFFFFF for kernel-thread
                                         */
    void                  *sysenter_return;
    struct restart_block   restart_block;

    unsigned long         previous_esp; /* ESP of the previous stack in case
                                         of nested (IRQ) stacks
                                         */
    __u8                  supervisor_stack[0];
};
```

Задание на 1-ю лабораторную работу «Загружаемые модули ядра»

Написать загружаемый модуль ядра, который при загрузке записывает в системный журнал информацию из кольцевого списка процессов. Перечень выводимых в syslog полей struct task_struct задается на семинаре.

Модуль должен собираться при помощи Make-файла.

Загружаемый модуль должен содержать:

- Указание лицензии GPL
- Указание автора

Примеры Make-файла предназначенного для сборки и компиляции загружаемого модуля ядра:

1. Makefile used to compile the kernel modules:

```
obj-m +=ProcessList.o
```

```
obj-m +=traceinit.o
```

```
all:
```

```
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
```

```
clean:
```

```
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

2.

```
ifneq ($(KERNELRELEASE),)
    obj-m := md.o
else
    CURRENT = $(shell uname -r)
    KDIR = /lib/modules/$(CURRENT)/build
    PWD = $(shell pwd)
default:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
clean:
    @rm -f *.o *.cmd *.flags *.mod.c *.order
    @rm -f *.*.cmd *~ *.*~ TODO.*
    @rm -fR .tmp*
    @rm -rf .tmp_versions
disclean: clean
    @rm *.ko *.symvers

endif
// $(MAKE) - вызов MAKE в режиме ядра.
```